Design Principles

Purpose

The Architectural Philosophy describes the fundamental beliefs that shape the approach to enterprise software architecture.

This document translates those beliefs into practical design guidance.

Design principles are not independent ideas. They are derived from the architectural philosophy and provide consistent direction for architectural decisions, service design, implementation, and engineering governance.

While technologies, frameworks, and programming languages inevitably evolve, these principles are intended to remain stable because they describe *how* architectural philosophy should influence software design.

Principle 1 — Operational Truth is Authoritative

Derived from:

- Reality Precedes Representation

Operational state should be modeled directly.

Documents, accounting transactions, reports, dashboards, notifications, and analytics are representations of operational reality rather than independent systems of record.

Architectural decisions should preserve a single authoritative operational model from which all representations can be derived.

Principle 2 — State Governs Behavior

Derived from:

- Reality Precedes Representation
- Governance is Superior to Correction

Business capabilities should be expressed as governed lifecycles.

Current state determines permitted operations.

Transitions are explicit.

Invalid transitions should be structurally impossible whenever practical.

Principle 3 — Design for Governance

Derived from:

- Governance is Superior to Correction

Governance should be embedded within the architecture itself.

Architectural constraints should prevent inconsistent states rather than relying primarily on downstream validation.

Validation reinforces governance.

It should not replace it.

Principle 4 — Preserve Optionality

Derived from:

- Optionality is an Architectural Asset

Architectural decisions should maximize future flexibility while minimizing unnecessary coupling.

Stable abstractions should allow implementation, interfaces, deployment models, automation, and presentation technologies to evolve independently.

Principle 5 — Separate Truth from Projection

Derived from:

- Reality Precedes Representation

Operational truth should remain independent of its consumers.

Reporting, accounting, document generation, integrations, notifications, analytics, and user interfaces should be implemented as projections over governed operational state rather than competing sources of truth.

Principle 6 — Contracts Precede Implementation

Derived from:

- Coherence is the Measure of Good Architecture

Architectural understanding should precede implementation.

Service boundaries, operational contracts, invariants, and lifecycle semantics should be established before implementation begins.

Implementation exists to realize architecture rather than discover it.

Principle 7 — APIs Express Operational Capabilities

Derived from:

- Preserve Optionality
- Reality Precedes Representation

The API surface represents operational capabilities rather than user interface workflows.

Stable service contracts enable multiple consumers—including user interfaces, integrations, automation, and AI agents—to interact consistently with the platform.

Principle 8 — Documentation Preserves Architectural Intent

Derived from:

- Coherence is the Measure of Good Architecture

Architecture should remain understandable beyond its original implementation.

Architectural decisions, design rationale, contracts, engineering governance, and technical documentation preserve intent and enable coherent long-term evolution.

Documentation is therefore considered an architectural asset rather than a project artifact.

Principle 9 — AI Amplifies Judgment

Derived from:

- Judgment is the Scarce Resource

Artificial intelligence should amplify architectural and engineering capability rather than replace human judgment.

AI is particularly valuable for implementation, documentation, synthesis, and analysis.

Architectural direction, abstraction, governance, and trade-off evaluation remain fundamentally human responsibilities.

Principle 10 — Coherence is Continuously Maintained

Derived from:

- Coherence is the Measure of Good Architecture
- Complexity Compounds

Architectural coherence is not achieved once.

It is maintained continuously.

As systems evolve, inconsistencies should be treated as signals that the underlying model requires refinement.

When conflicts arise, preference should be given to improving the conceptual model rather than introducing isolated exceptions.

Good architecture evolves by preserving coherence rather than accumulating accommodation.

Relationship to the Portfolio

These principles provide the interpretive layer between architectural philosophy and implementation.

Every major architectural decision documented within the Gildmark portfolio should be explainable as the application of one or more design principles.

Likewise, every design principle should be traceable to one or more philosophical beliefs documented in the Architectural Philosophy.

This hierarchy ensures that implementation remains coherent because it is consistently derived from stable underlying concepts rather than isolated design choices.